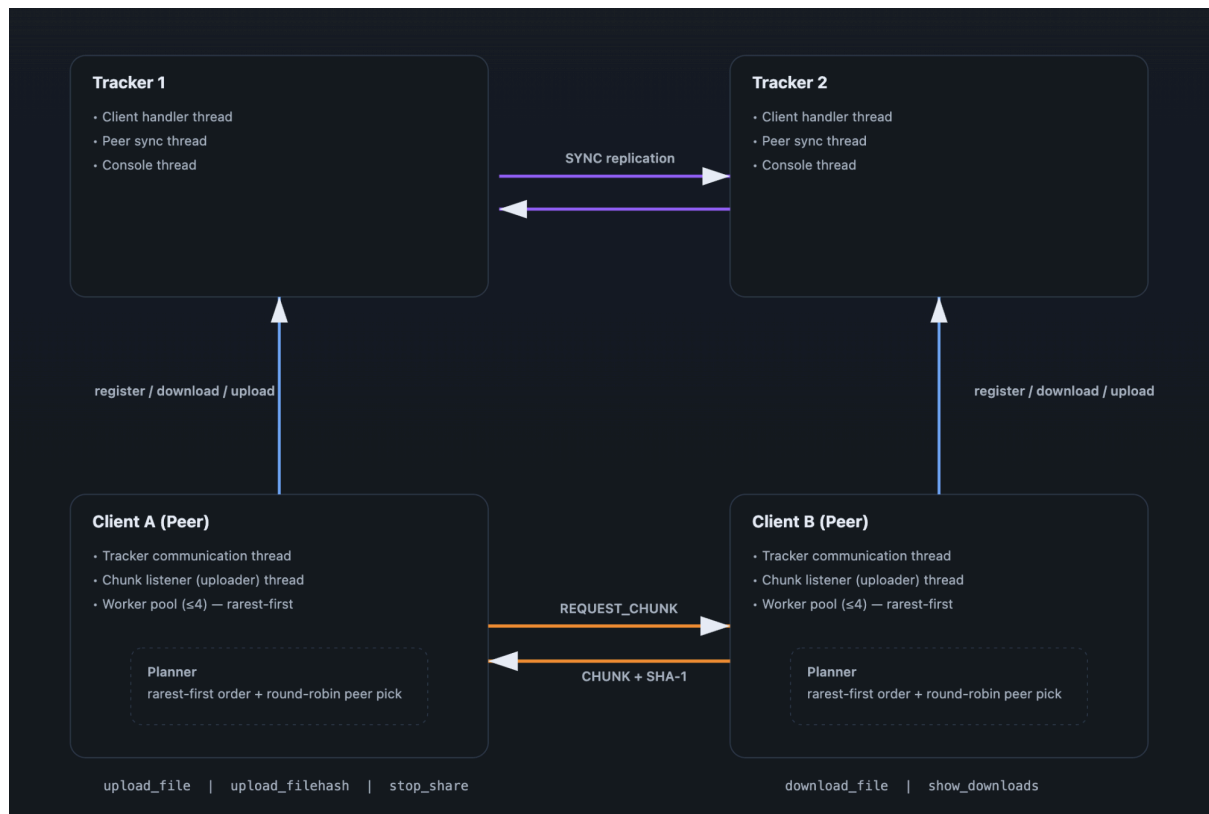


Introduction

The goal of this assignment was to design a P2P file sharing system which will allow the clients to download the files from the peers without a centralized file server. The system takes help of hybrid architecture where we are storing only the metadata on the tracker and use peers for decentralised file transfer. Here we have used 2 trackers so that if one of the trackers goes down still the system will work.

Primary intent behind the system was to distribute the file into fixed chunks of 512 KB so that parallel downloading can happen from multiple peers and hence faster download. Also due to 2 trackers we are eliminating a single point of failure. We have also ensured that chunks are verified when they are downloaded hence we are checking the SHA based integrity. We have also introduced the rarest first chunk selection so that the chunk which is with less number of peers will be downloaded first and for load balancing we are using round robin technique so we are not fetching all the chunks from the same peer.

System architecture overview



Here we are following hybrid architecture which balances between the centralised coordination with fully decentralised data transfer. It has 2 parts :

1. **Trackers** : These are lightweight coordination servers which just maintain very basic data like it maintains file's metadata, peer's availability, group details, list of which clients have accounts, and maintains current active sessions with the tracker. Tracker will not store any actual data of the file it will just store the metadata and it will also store the full file SHA1.
2. **Peers** : This will act as client when it is communicating with the server like when it wants to login, make groups and all then it will act as client and the tracker will act as

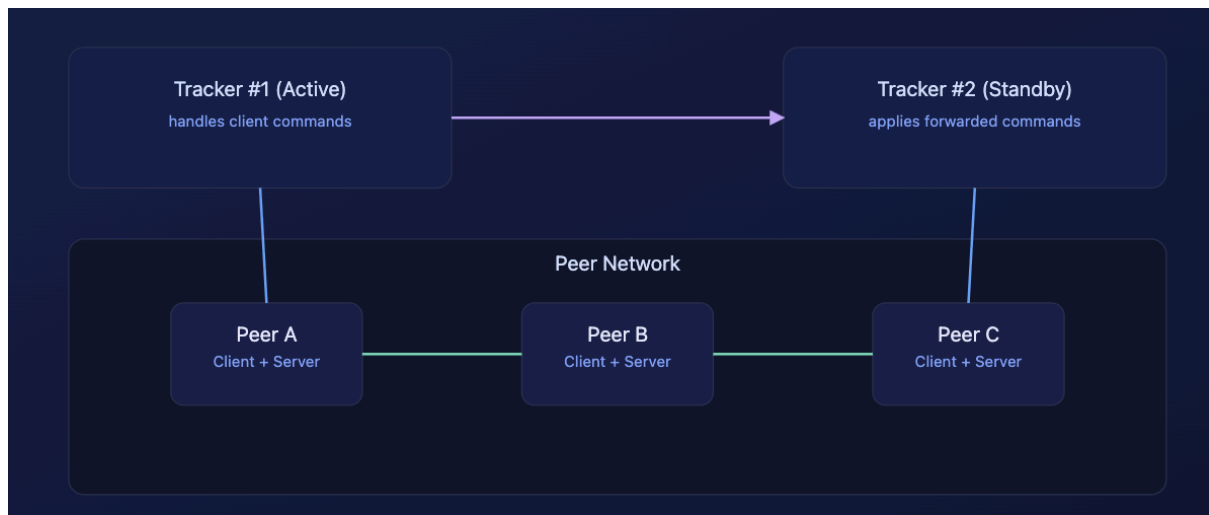
server and when the downloading of file is happening then the peer who is sharing the file will act as the server in that case and the peer who is downloading the file will act as client in that case.

The entire system is built over TCP sockets because :

1. It guarantees in-order delivery.
2. It also has built-in congestion handling and retransmission.
3. It provides full duplex streaming support so it will enable simultaneous uploads and downloads.

When peer is sending commands to the tracker like `create_user`, `login`, `create_group` it will use TCP and also the communication in the peer channel is TCP where we are using commands like `REQUEST_CHUNK`, `CHUNK <size> <sha>` raw data, this also uses TCP connection.

Tracker Design and synchronisation



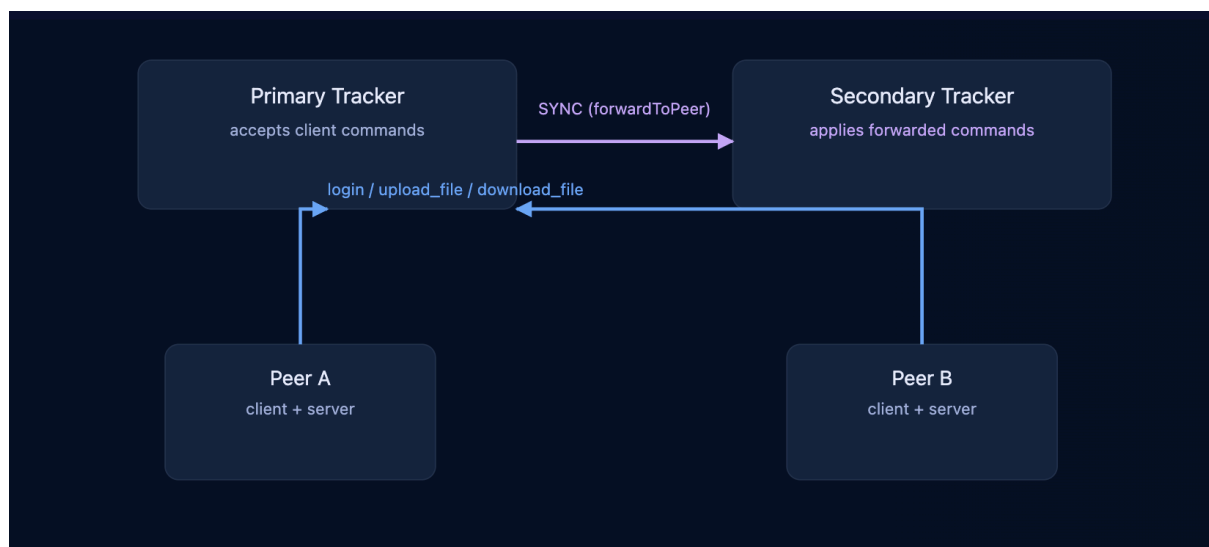
Tracker maintains multiple data structures like `map<string,UserInfo> userInfo` which stores the registered users, their credentials like password, their login status, its ip and port which will be used when other peers want to communicate with this peer. `map<string, string> groupOwners` this will store which user owns which group, `map<string,set<string>> groupMembers` which will store which peers are part of which group, `map<string,map<string>> activeSessions` this will map client socket to user id relationship, `map<string,map<string, FileInfo>> fileTable` this will store which group has which file and then information related to that file will be stored like which peer has which chunk and full file SHA. These structures are shared across multiple client-handling threads so synchronisation in this case is critical.

For concurrency we are using some mutex. Each incoming client connection is served in its own thread and since all threads operate on shared data we need to protect parallel data access to avoid any race conditions or inconsistencies and we do this using mutex. Different mutexes which we are using are `sessionMutex`, `userInfoMutex`, `groupsMutex`, `fileTableMutex`.

We are using fire and forget replication strategy to implement tracker synchronisation. Whenever there is some command on the main tracker which will change the shared data then we will forward that command to the peer tracker and it will just apply that command on its end as well so that data in these trackers are synchronised. We are not waiting for any acknowledgement, we are assuming that it will apply on its end. We are following this strategy under the assumption that if one tracker goes down then it won't come back again and also initially both the trackers are one, not like if one tracker is up after some state change then there will be tracker synchronisation. Here as there is low command latency hence we will get consistency immediately before we enter the new command. Commands are applied in the same order they appear on the main tracker; there is no parallel write for the commands on the peer tracker. If one of the tracker fails then another tracker will stop sending the sync commands and it will take over as the primary tracker, this leads to one good fault tolerant system.

We created one thread called `keepPeerConnected` which will try connecting to the peer tracker. We also created one more thread which will listen to the inputs from the stdin and whenever the quit command is found it will send a SIGINT signal to the tracker and hence the tracker shuts down and the peers will be asked to connect to another tracker.

Client - Tracker Communication



We are sending commands from the client to the tracker in one single line and then on the tracker the tracker will handle these commands and then update the shared data accordingly and if it is something like a download command then it will respond with appropriate data.

On the tracker side there is `while(true)` loop which will always listen for connections from the tracker and once the client send the request to connect to the tracker then it will create one thread per client and in this thread we will run the function called as `handleClientCommands` and hence all the commands received from the client will be handled differently for each client.

Client command send format : `login <userId> <password> <clientIp> <clientPort>\n`

We are checking multiple cases like if the user is not logged in then they won't be able to create_group or won't be able to download or upload the file. Appropriate errors are sent to the client so that the client can modify the command and send the required format of the command.

Let's say that recv() on the client side fails then the client closes the socket and it will call connectToTracker(..) with the 2 trackers information and if it successfully connects then it resumes the working again or else it will just terminate the program. Here sending and receiving of the commands happen on the same thread because we don't want parallelism and we want the commands to be executed in the same order they were sent in.

On the tracker side there is one thread per client connection (trackerThread) and in the thread we will also be deciding whether some command is a SYNC command or not and if it is a SYNC command then we will send it to the peer client or else it will be applied in the same tracker. We are using different locks in this case because if many clients send commands concurrently then the tracker stays consistent via these locks and we can be assured that the data is valid.

Upload command : upload_files <groupId> <fileName> <fileSize> <numOfChunks> <clientIp> <clientPort> first this command is sent to the tracker which will then update the file information on the tracker side. After this client will send one more command upload_filehash <sha1hex> this will upload the SHA of the full file onto the tracker

Download command : When client sends this command to the tracker then the tracker replies to the client with all the meta data which is required to request downloading of file from the seeder.

```
FILEINFO video.mpy 12710019 25
PEER 127.0.0.1 1234 0,1,2,3,4,5
PEER 127.0.0.1 1235 6,7,8,9,10
END
```

For each and every communication we are using TCP whether it is client to tracker, tracker to tracker or client to client communication.

Peer - Peer Communication and Transfer



This communication is used only when we want to download the file, like client B wants to download the file from client A then there will be a link between client A and B and once the download is completed then this will be broken. The goal of this is to efficiently transfer large files split into fixed size chunks of 512 KB between peers while maintaining integrity and parallelism.

Explanation in short : We transfer fixed-size chunks over TCP using a small text header `CHUNK <size> <sha1>` plus raw bytes. Download order is determined by a tracker-informed rarest-first policy to improve availability. A small worker thread pool (atomic task index) parallelizes download where each worker fetches a chunk from a chosen peer, verifies per-chunk SHA-1, and writes to the file with the `pwrite()` (no write mutex required as this is). Failures are handled by retries and falling back to other seeders for that chunk; after all chunks are done we optionally verify the full-file SHA stored by the tracker.

We have used TCP for reliable delivery and to detect corruption of data we are verifying SHA-1 for every chunk and once we receive a complete file then we are checking the SHA-1 value of the complete file to ensure that we got the correct file. Whenever downloading is taking place then we will schedule pieces in rarest first format so that we maximise the availability and reduce hot-spots. Also we will use a small thread pool of size 4 due to which we can download the chunks in parallel.

1. First client will send to tracker that it wants to download the file then it will give all the meta data related to the file
2. Then from the tracker's response it will get the ip and port of the seeder's then the client will send `REQUEST_CHUNK <filename> <chunkIndex>` to the seeder
3. On the seeder size if the chunk is present then the seeder will reply to the peer with `CHUNK <size> <sha1hex>` followed by raw data `<= 512 KB`
4. If the file downloading is unsuccessful then the command will report ERROR

Integrity check : Seeder computed the SHA-1 for the chunk and it will send this to the requester with the `CHUNK` and the requester will calculate the SHA-1 of the chunk which it has received and checks whether they match or not, if they mismatch then it is corrupted and we won't write this chunk to the file. Why are we doing this? Per-chunk hashes let us

detect corruption early and the final file hash detects whether the file is the same or accidentally the seeder send the wrong file.

When we are sending and receiving the 512 KB data then we are using 2 different custom functions called as `sendAll()` and `recvAll()` these are helpers which will loop until the requested amount that is 512 KB is transferred and we are using TCP and it doesn't ensure that all the bytes are sent in one call and we want to receive and send in blocks of 512 KB so in this helper we will keep calling `send()` or `recv()` till we get or send requested amount of data.

When downloading there are multiple peers which have the same chunk and there are multiple chunks so we need to plan which chunk to be requested first and from which peer we need to get it from so for choosing which chunk to be requested first we are using the rarest-first piecewise selection strategy. We are using this because it reduces the chance that a chunk becomes unavailable if many peers leave mid-download. It helps distribute the load by prioritising less-available pieces.

To decide which peer to get this from we are using a round-robin strategy so this will help to distribute the load on multiple peers and so that we don't end up fetching all the chunks from the same peer or else there is no use of the P2P architecture. Also we first check whether the given peer is online or not, if it is offline then we don't request chunk from that peer.

We are using the concept of thread pooling at the requester side to efficiently fetch chunks of data in parallel without spawning an unbounded number of threads. Instead of creating a new thread for every chunk we are fixing the number of worker threads to 4. These workers will pick the tasks from the jobs from a shared atomic counter using `(nextTask.fetch_add(1))`. Each task in my code specifies which chunk to fetch and from which peer. Then worker thread will connect to that peer and send `REQUEST_CHUNK <file> <chunkIndex>` and it receives the chunk bytes and head, it verified integrity via SHA-1 and then writes it at the correct offset in the destination file using `pwrite()` instead of `write()` as it is thread safe. If one worker finishes early then it automatically picks up the next available task ensuring load balancing across threads. This avoids idle workers while keeping concurrency controlled. By limiting workers to only 4 we are reducing the context switch overhead and avoiding exhausting CPU or network resources while still overlapping multiple network IO operations to maximise the throughput. We can increase the number of threads up to CPU cores, my system is Macbook M4 air it has 10 cores, on this machine we could use up to 16 threads but this will reduce the efficiency.

Show Downloads:

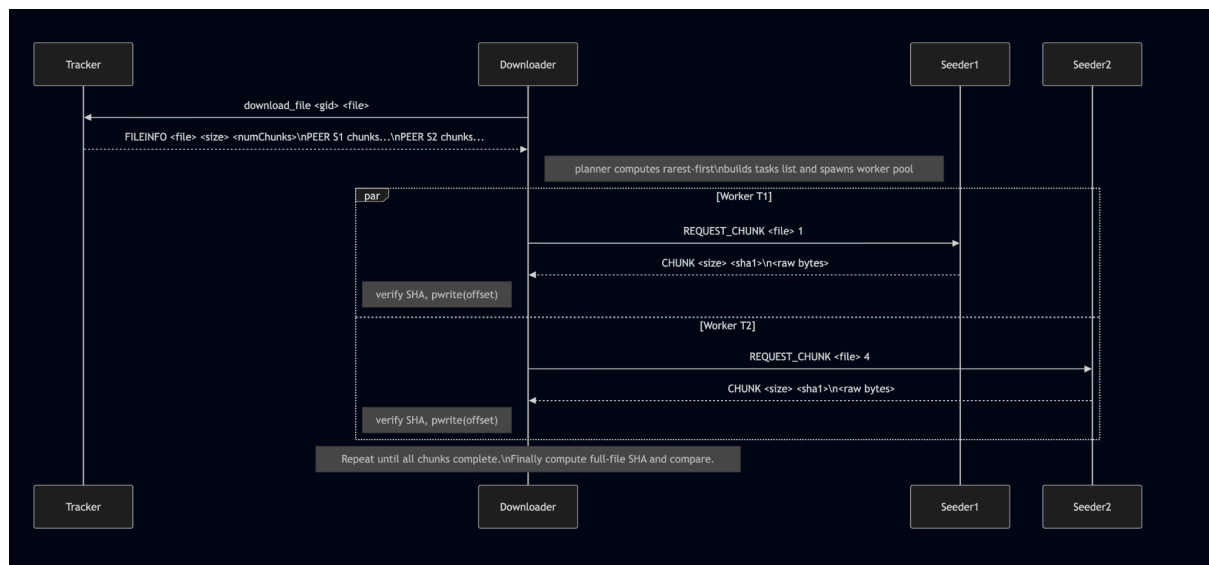
The `show_downloads` command allows a peer to track the progress of ongoing and completed file downloads. Each file being downloaded is represented with metadata such as its total number of chunks, how many chunks have been successfully received, and whether the download is still active, failed, or completed. The system iterates over its local `downloads` data structure and prints a detailed status line for every file, showing chunk counts and percentage progress. This feature helps the user quickly verify if a large file is still in progress or has finished downloading successfully.

Stop Share:

The `stop_share` command enables a peer to stop sharing a file it had previously uploaded

to a group. When invoked, the peer notifies the tracker, which removes the peer's association with that file from its metadata tables. This means new downloaders will no longer see that peer as a source for the file's chunks. If the peer was the only seeder, the file effectively becomes unavailable until another peer shares it again. This ensures peers maintain control over which files they contribute to the swarm, balancing availability with user privacy and resource management.

Sequence diagram how data flows :



Testing and Performance Analysis

```

>> download_file g1 Croods.jpg "/Users/srushtipekamwar/Desktop/croodsDesktop.jpg"
Destination: /Users/srushtipekamwar/Desktop/croodsDesktop.jpg
CHUNK HASH VERIFIED for index = 0
[Downloader] Wrote chunk: 0 Source: 127.0.0.1:1234
[Verifier] File hash OK: de76c972bd12227d755ec5082e5e8b399488fd80
[Planner] Download completed successfully.
[Planner] Elapsed time: 0.025 s, throughput: 15.825 MB/s
[Planner] Registered local path for Croods.jpg -> /Users/srushtipekamwar/Desktop/croodsDesktop.jpg
Notified tracker about upload_file g1 Croods.jpg 408330 1 127.0.0.1 2357

```

```

[Planner] Download completed successfully.
[Planner] Elapsed time: 3.133 s, throughput: 260.327 MB/s
[Planner] Registered local path for groundFloor.mp4 -> /Users/srushtipekamwar/Desktop/groundDesktop.mp4
Notified tracker about upload_file g1 groundFloor.mp4 855159381 1632 127.0.0.1 2357

```

First image is when we tested on a small scale file of only 1 chunk which was of 408 KB. We were able to download the file successfully with per chunk SHA and full file SHA verification with negligible overhead. The total elapsed time was 0.025 sec and the effective throughput was 15.825 MB/s. Elapsed time means time taken from the moment you start downloading a file until the file is fully received and verified.

In the next image we tested on a large file of around 1632 chunks and the file size was 855.2 MB. We were able to download the file with integrity verification for every chunk and the full file. Elapsed time was 3.133 s and effective throughput was 260.327 MB/s. The results indicate that the system achieves very high throughput, benefitting from parallel chunk fetching with worker thread pool and low synchronisation overhead.

With 4 threads-pool, chunks were fetched concurrently from multiple peers, which maximized utilization of network bandwidth and reduced idle waiting. If parallel download had not been implemented and instead it was single thread or sequential fetch then the downloader would have requested chunks one by one from the peer and the total time would scale roughly linearly with number of chunks. For the larger file the elapsed time would likely have been several times slower, depending on latency and peer availability. Hence parallel scheduling is critical for achieving high throughput.
